



Leveraging ParSEC for Task-Based Heterogeneous Computing in Python and Julia

Qiao Zhang¹, Rabab Alomairy^{2,3}, Dali Wang⁴, Addison Thurston¹,
and Qinglei Cao¹(✉)

¹ Saint Louis University, St. Louis, USA
qinglei.cao@slu.edu

² Massachusetts Institute of Technology, Cambridge, USA

³ King Abdullah University of Science and Technology, Thuwal, Kingdom
of Saudi Arabia

⁴ Oak Ridge National Laboratory, Oak Ridge, USA

Abstract. We present Python and Julia bindings (PyParSEC and ParSEC.jl) for the ParSEC task-based runtime system, providing high-level access to its efficient and portable execution model on distributed heterogeneous architectures. The bindings expose ParSEC’s core abstractions—including data management, task creation, and runtime execution—allowing domain scientists to construct parallel workflows from high-level code while preserving ParSEC’s execution semantics. We evaluate these bindings using a 1D stencil workload (through ParSEC’s Parameterized Task Graph (PTG)) and a general matrix–matrix multiplication (GEMM) workload expressed via Dynamic Task Discovery (DTD). The results show that the bindings reproduce the official workflows with modest overhead, and that performance remains close to native implementations when leveraging optimized numerical backends on CPUs and vendor-provided accelerator libraries on GPUs, while callback-based user kernels provide flexibility at higher overhead. This work demonstrates that ParSEC’s high-performance runtime can be effectively leveraged from productive, high-level languages, bridging the gap between scientific prototyping and exascale execution.

Keywords: High-performance computing · Task-based runtime · Heterogeneous computing · Python · Julia

1 Introduction

Modern high-performance computing (HPC) applications increasingly run on heterogeneous platforms that combine multi-core CPUs with one or more accelerators, and they often scale to distributed clusters. Achieving high performance therefore depends not only on fast kernels, but also on exposing concurrency, managing data movement, and adapting scheduling to system conditions. Task-based runtime systems address these challenges by representing computation as

a task graph with dependencies, enabling dynamic scheduling and overlap of communication and computation.

PaRSEC [1, 2] is a widely used task-based runtime for distributed and heterogeneous execution. It couples dependency-driven scheduling with tiled, locality-aware data management, and provides multiple domain-specific interfaces to express task graphs: Parameterized Task Graph (PTG) [3] and Template Task Graph (TTG) [4] for structured workloads, and Dynamic Task Discovery (DTD) [5] when the task graph must be constructed at runtime. PaRSEC is primarily accessed through C/C++ interfaces, which raises the barrier for workflows increasingly written in Python and Julia [6–9]. Bridging high-level productivity languages to performance-oriented task runtimes is thus important to scale prototypes without rewriting the workflow [10].

In this work, we present Python and Julia bindings for PaRSEC that provide high-level access to PaRSEC’s core abstractions—context management, tiled data collections, task creation, and device execution—while preserving the execution semantics of the official PaRSEC examples. The Python interface (PyPaRSEC) targets integration with array-centric workflows and enables users to drive PaRSEC task graphs from familiar data structures. The Julia interface (PaRSEC.jl) leverages Julia’s composability to express dynamic task graphs naturally while mapping them to PaRSEC’s runtime mechanisms. Our design follows a simple principle: keep the programming model high-level, while keeping the critical runtime objects and execution paths close to PaRSEC’s native representations to minimize overhead. The contributions are as follows:

- We design and implement Python and Julia APIs for PaRSEC, exposing core runtime abstractions (context, taskpools, task classes, and tiled data collections) to high-productivity languages while preserving the execution semantics of official PaRSEC examples.
- We support the PTG templates in both bindings. Python/Julia acts as a thin driver that configures contexts and tiled data collections and launches existing native PaRSEC taskpools, preserving the execution semantics and kernels of the official PTG examples.
- We implement DTD support in both PyPaRSEC and PaRSEC.jl, enabling dependency-aware task classes and scalable runtime task insertion over tiled iteration spaces; for heterogeneous execution, performance-critical compute is delegated to optimized native C GPU kernel wrappers while PaRSEC manages scheduling and data movement.
- We evaluate the bindings using two representative workloads—stencil (PTG) and DTD GEMM—against the corresponding official C baselines under consistent configurations, and we discuss callback-based user kernels as an optional feature that improves flexibility at higher overhead.

Together, these results demonstrate that PaRSEC’s high-performance runtime can be effectively leveraged from productive high-level languages, enabling a more direct path from scientific prototyping to exascale-capable execution.

2 Related Work

2.1 Task-Based Runtimes for HPC

Task-based runtime systems extract parallelism from complex workloads by expressing computation as a directed acyclic graph (DAG) of tasks with explicit data dependencies. Compared with bulk-synchronous execution, dependency-driven scheduling enables dynamic concurrency discovery, overlap of communication and computation, and better tolerance to load imbalance, which is increasingly important on heterogeneous CPU/GPU platforms at scale.

Most task runtimes provide (i) a mechanism to represent dependencies and task readiness, (ii) a scheduler that maps ready tasks to CPU cores and/or accelerators, and (iii) a data management layer that tracks the location and lifetime of task inputs/outputs. In HPC, tasks are often defined over tiled (block-partitioned) data to expose fine-grained parallelism and to make locality-aware scheduling effective.

Representative systems include OpenMP [11], OmpSs [12], COMPSs [13], StarPU [14], HPX [15], and Legion [16]. While their interfaces differ, they share the goal of letting applications describe *what* work and dependencies exist, while the runtime decides *where* and *when* tasks execute. Despite their performance potential, these runtimes are most commonly accessed through low-level C/C++ interfaces, motivating high-level language bindings that improve programmability while preserving efficient interaction with runtime scheduling and data management.

2.2 High-Level Language Bindings and Interoperability

Building on these runtime capabilities, a natural next step is to expose such execution models to productive high-level languages. Several efforts have exposed task-based runtimes to high-level languages to improve programmability. Examples include Python front-ends and bindings for existing runtimes (e.g., StarPU’s Python interface) [17] and language front-ends to HPC task runtimes such as Pygion for Legion [18]. A common challenge in such bindings is bridging the mismatch between runtime expectations and high-level language semantics: runtimes typically expose low-overhead C interfaces and often require callers or the binding layer to ensure argument/data-handle lifetimes across asynchronous, multi-threaded execution, while high-level languages introduce garbage collection, exception semantics, and (for Python) the Global Interpreter Lock (GIL). These issues complicate safe and efficient kernel invocation and data-handle management.

Interoperability with array/tensor ecosystems is another practical concern. In Python, libraries such as CuPy and interoperability mechanisms such as DLPack facilitate exchange of GPU-resident arrays between libraries, but performance is sensitive to data residency and transfer behavior. Without careful integration, per-task host-device copies and synchronization can dominate runtime, obscuring compute performance. These observations motivate designs where the

binding layer keeps control-plane logic in the high-level language while keeping performance-critical execution paths close to runtime’s native representations.

3 Background

3.1 PaRSEC Runtime System

PaRSEC is a task-based runtime framework for scheduling and managing micro-tasks on distributed, multi-core, and heterogeneous architectures. It combines (i) a dependency-driven dataflow runtime, (ii) multiple programming interfaces for defining task graphs, and (iii) auxiliary components that ease integration with existing HPC software stacks and support performance analysis and tuning [19]. PaRSEC has been used as a runtime substrate for a broad range of HPC workloads, including dense linear algebra and more irregular algorithms [20–24], in various applications [25–27]. In practice, PaRSEC commonly executes tasks over tile/block-partitioned data. Its data management layer tracks the location and state of these tiles and enforces dependencies at tile granularity, which is critical on heterogeneous systems for coordinating task placement with data locality and orchestrating transfers between host and accelerator memories. PaRSEC provides several domain-specific interfaces to express task graphs, including the PTG, TTG, and DTD. PTG and TTG capture regular task structures compactly, while DTD enables tasks and dependencies to be created dynamically at runtime, which is particularly suitable for irregular control flow and for high-level language front-ends. Our work builds on PaRSEC’s runtime strengths and focuses on language accessibility: we expose PaRSEC’s execution model to Python and Julia users while preserving workflow parity with official examples and maintaining a thin binding layer. In this paper, we focus on implementing the PTG and DTD interfaces in Python and Julia bindings.

3.2 Julia Programming Language

Julia is a high-level programming language for scientific and technical computing that aims to combine interactive productivity with performance close to traditional compiled languages. Its design emphasizes composable abstractions and compiler specialization, and it provides native support for shared-memory multi-threading and distributed execution. The Julia ecosystem has also matured into a practical platform for HPC and data-intensive workloads, with representative applications spanning scientific computing [28], scientific machine learning [29], data science [30], optimization [31], and simulations [32].

4 Implementation Details

4.1 API Overview and Execution Paths

PyPaRSEC and PaRSEC.jl expose PaRSEC concepts through a thin, explicit object model. The bindings currently target two primary execution paths that

correspond to ParSEC’s native programming interfaces: PTG and DTD. In PTG tasks, the high-level language acts mainly as a driver that configures runtime contexts and data collections and launches an existing ParSEC taskpool; the task graph and kernels are provided by ParSEC. In DTD tasks, the binding exposes DTD-specific context and taskpool abstractions, allowing users to define task classes, insert task instances at runtime, and rely on ParSEC to enforce dependencies and schedule tasks as data becomes available [5]. To address the common challenges in Sect. 2.2, both bindings expose ParSEC contexts, taskpools, and data collections as explicit objects backed by stable C pointers, and materialize DTD VALUE arguments in contiguous C-backed buffers that are kept alive beyond the insertion call because ParSEC stores and later dereferences these pointers at execution time. For GPU tasks, performance-critical computation is delegated to native C kernels, while callback-based kernels are provided as an optional path to improve user flexibility.

4.2 PTG Task

For PTG-based applications, the bindings support executing existing ParSEC workloads whose task graphs and kernels are defined in the PTG framework. In this mode, Python/Julia provides a thin driver layer: it creates a standard ParSEC context, configures the required tiled data collections (including halo/ghost regions when applicable), instantiates the corresponding ParSEC taskpool, launches execution, and finalizes. Since the task graph structure and the performance-critical kernels are the same as in the official ParSEC PTG implementation, this path isolates the overhead of the high-level driver and binding layer while reusing the optimized native PTG kernels.

4.3 DTD Task Insertion and Kernel

For dynamic workloads, our goal is to allow Python and Julia users to construct ParSEC task graphs through the DTD interface with the same workflow structure as the official DTD examples, while keeping performance-critical execution in optimized backends.

From the Python/Julia side, the DTD workflow follows four steps: (1) initialize and start the ParSEC DTD context; (2) initialize memory and data; (3) define a task class with explicit data dependencies and dynamically insert task instances; (4) flush data collections and wait for completion.

4.4 Callback-Based User Kernels

In addition to the performance-oriented execution paths, our bindings optionally support callback-based user kernels written in Python or Julia. In this mode, tasks are configured to invoke a user-provided callable when they become ready, so users can express per-tile computations directly in the high-level language. This enables rapid prototyping and convenient integration with existing numerical ecosystems: the callback can call standard Python/Julia library functions

on the input tiles, without requiring users to implement PaRSEC-specific native kernels. In our current prototype, callback execution is supported in PyPaRSEC on CPUs and NVIDIA GPUs, and in PaRSEC.jl on CPUs; callback support for additional accelerator backends like AMD GPUs is not yet provided. However, callback execution introduces additional overhead compared to native kernels, including language runtime transitions and synchronization constraints. As a result, we use callbacks primarily for functional validation and flexibility, while our performance evaluation focuses on optimized compute backends to keep the critical path efficient.

5 Performance Results and Analysis

5.1 Experimental Setup and Methodology

The experiments are conducted on the following systems:

- Leconte (CPU): CPU-only experiments are run on an Innovative Computing Laboratory (ICL) Leconte node with dual-socket Intel Xeon E5-2698 v4 (Broadwell-EP) processors (40 physical cores, 80 hardware threads with SMT). GPUs are not used in this configuration.
- Hexane (CPU): CPU-only experiments are run on the ICL Hexane node, which is powered by dual-socket Intel Xeon Silver 4309Y processors (8 cores per socket with SMT, 16 physical cores and 32 hardware threads total) at 2.80 GHz. GPUs are not used in this configuration.
- Leconte (GPU): We run the GPU-accelerated experiments on the same Leconte node, which has eight NVIDIA Tesla V100-SXM2-32GB GPUs. The host CPUs remain the dual-socket Intel Xeon E5-2698 v4 (40 cores total), while the compute-intensive kernels execute on the V100 GPUs.
- Hexane (GPU): This is a GPU-accelerated Intel compute node at ICL. It is equipped with dual-socket Intel Xeon Silver 4309Y CPUs (8 cores per socket with SMT, 32 hardware threads total) at 2.80 GHz, and two NVIDIA GPUs: an NVIDIA H100 PCIe and a Tesla V100-PCIE-16GB.
- Polaris: It is a Cray-built HPC system. Its GPU-accelerated compute nodes are equipped with a single-socket AMD EPYC 7543P CPU and four NVIDIA A100-SXM4 accelerators, each with 40 GB of HBM2 memory.

We evaluate the overhead and performance of PaRSEC bindings in two high-level languages, Python and Julia, using two representative workloads that exercise the two main execution paths exposed by our bindings: 1D stencil through PaRSEC’s PTG, and tiled DTD GEMM through DTD. These workloads serve as case studies to validate that the bindings reproduce the official PaRSEC workflows and can achieve competitive performance when paired with optimized compute backends. Our evaluation reports three categories of timing results:

- (1) PaRSEC initialization cost.
- (2) Task-graph construction cost (DTD task insertion only).

(3) End-to-end execution time for representative workloads.

For (1), we summarize the PaRSEC runtime bring-up overhead in a table, comparing the native C drivers against PyPaRSEC and PaRSEC.jl. We measure the time from creating the runtime context to reaching a ready-to-run state after starting the runtime. This metric excludes task insertion and kernel execution.

For (2), we report the cost of DTD task-graph construction (task insertion) in a separate table. To isolate insertion overhead from computation, we disable all compute kernels in this microbenchmark and vary only the number of inserted task instances. The timing window starts immediately before inserting the first task and ends after all insertions have been flushed to the runtime.

For (3), we measure end-to-end execution time for stencil and GEMM workloads, including both scheduling and computation. Each experiment is executed three times under the same configuration, and we report the minimum observed runtime across runs. This choice reduces the impact of transient system noise and reflects the attainable performance of the runtime and implementation under stable conditions.

For stencil, we execute the official PTG stencil interface, with the high-level language acting as a driver to create the standard PaRSEC context, set up data collections, launch the taskpool, and finalize; we report end-to-end time over 1000 iterations with radius $R = 1$. For DTD GEMM, we use PaRSEC’s DTD workflow and a tiled formulation over the (m, n, k) iteration space with explicit data dependencies. On CPUs, tile GEMM is performed through BLAS-backed high-level kernels (NumPy for Python and LinearAlgebra for Julia), while the official C baseline invokes optimized BLAS routines directly. On GPUs, both our binding-driven implementation and the official baseline use PaRSEC’s native GPU execution path with cuBLAS-based kernels. Unless otherwise stated, all the tests use a square tile size of 1024×1024 .

Across all experiments, we keep problem sizes, tiling parameters, iteration counts, and runtime settings consistent to ensure fair comparisons. In all bar charts, annotations indicate the relative execution-time difference of PyPaRSEC/PaRSEC.jl with respect to the C baseline; negative values mean PyPaRSEC/PaRSEC.jl is faster.

5.2 Python API Evaluation

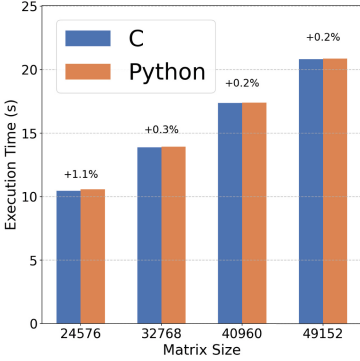
Initialization Overhead. We first report runtime initialization overhead in Python. Table 1 summarizes the results over 96 repeated runs for both the official C implementation and PyPaRSEC (mean $\pm$ std). Under the stencil configuration, PyPaRSEC’s initialization time is 1711.220 ± 53.246 ms, which is nearly identical to the official C implementation at 1703.360 ± 33.973 ms; the additional overhead of Python relative to C is only 0.46%. This result indicates that exposing PaRSEC to Python does not significantly increase the runtime bring-up cost.

Table 1. Initialization time comparison on Leconte CPU node (mean $\pm$ std over 96 runs, ms).

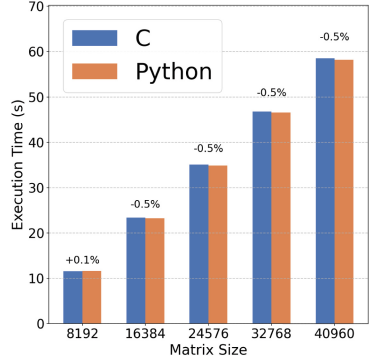
Workload	C (ms)	Python (ms)	Python overhead
Stencil	1703.360 $\pm$ 33.973	1711.220 $\pm$ 53.246	+0.46%

Stencil. We evaluate stencil through PaRSEC’s PTG workflow, where Python acts as a driver that configures data collections and launches the existing taskpool. We report end-to-end execution time for the stencil workload.

Figure 1 compares PyPaRSEC against PaRSEC’s official C stencil example on two CPU nodes. Across all problem sizes, PyPaRSEC matches the C baseline closely: the relative differences remain within $\pm 1.1\%$ on both Leconte and Hexane. The execution time scales monotonically with problem size for both implementations, and the small positive/negative deviations fluctuate around zero, suggesting that the observed gaps are dominated by normal system variability rather than binding overhead. These results indicate that, for stencil workloads following PaRSEC’s PTG tasks, the Python binding acts primarily as a thin driver layer and preserves the performance characteristics of the native runtime.



(a) Leconte CPU node.



(b) Hexane CPU node.

Fig. 1. End-to-end execution time of PyPaRSEC compared with PaRSEC’s official C stencil example on CPU.

DTD GEMM. Before presenting end-to-end performance, we isolate the overhead of DTD task insertion, which captures the control-plane cost of constructing the dynamic task graph from the high-level language. To avoid any interference from kernel execution, we disable the compute kernels in this microbenchmark and only measure the time spent inserting tasks into the DTD taskpool.

Table 2 reports the mean and standard deviation of the insertion time on the Leconte node for three representative task-graph sizes. For each task count, we repeat the experiment 20 times for C, PyPaRSEC and PaRSEC.jl. We discuss Julia results in the next subsection. From the table, at small task-graph sizes PyPaRSEC incurs less than 30% additional insertion overhead compared to the C baseline. Given the unavoidable costs of crossing the language boundary, this gap is generally acceptable. However, Python overhead grows with the number of inserted tasks: at 110,592 tasks, PyPaRSEC incurs an additional $6.6\times$ insertion overhead over C, indicating that task insertion becomes increasingly dominated by per-task language-level overhead at larger scales.

Table 2. DTD task insertion overhead comparison on Leconte (mean $\pm$ std over 20 runs). The column headers (64, 4096, 110592) denote the number of tasks in the constructed task graph.

	64	4096	110592
C insert (ms)	0.640 ± 0.049	12.417 ± 0.212	281.922 ± 1.883
C ($\mu\text{s}/\text{task}$)	10.003 ± 0.773	3.032 ± 0.052	2.549 ± 0.017
Python insert (ms)	0.829 ± 0.092	53.650 ± 12.019	2142.762 ± 136.439
Python ($\mu\text{s}/\text{task}$)	12.959 ± 1.437 (+29.55%)	13.098 ± 2.934 (+332.06%)	19.375 ± 1.234 (+660.05%)
Julia insert (ms)	1.283 ± 0.062	68.420 ± 30.067	1570.917 ± 170.853
Julia ($\mu\text{s}/\text{task}$)	20.048 ± 0.967 (+100.42%)	16.704 ± 7.340 (+451.01%)	14.205 ± 1.545 (+457.22%)

For GPU DTD GEMM, Python drives the workflow and task insertion, while the compute kernels use PaRSEC’s native GPU execution path with cuBLAS. This experiment evaluates whether the binding can launch accelerator-enabled workflows with minimal additional overhead beyond the official baseline.

Figure 2 compares PyPaRSEC against PaRSEC’s official DTD GEMM C example on three GPUs: V100 (Leconte), A100 (Polaris), and H100 (Hexane). On the V100, PyPaRSEC is slower than the C baseline for all tested sizes, with a larger gap at the smallest size (+22.4%) and single-digit overheads at larger sizes. On the A100, the results are consistently close: the relative differences stay near zero at smaller sizes and remain within a few percent at larger sizes. On the H100, the relative difference is size-dependent; at the largest size tested, PyPaRSEC is about 6.7% slower than the C baseline. Overall, PyPaRSEC follows the same scaling trend as the C baseline across all platforms, while the relative gap varies across GPU systems and problem sizes.

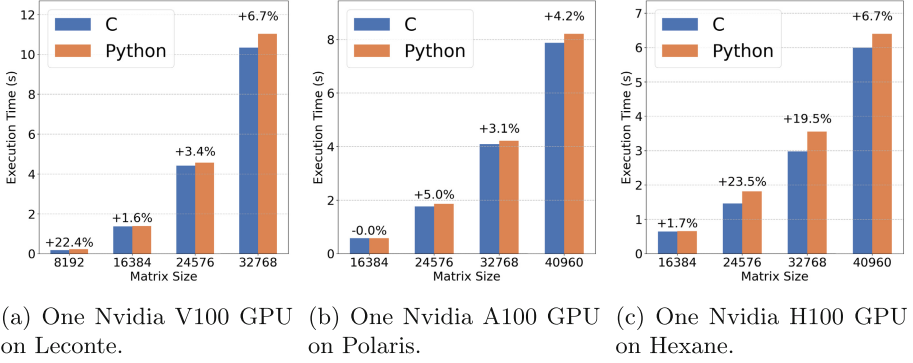


Fig. 2. End-to-end execution time of PyPaRSEC compared with PaRSEC's official C DTD GEMM example on GPU.

5.3 Julia API Evaluation

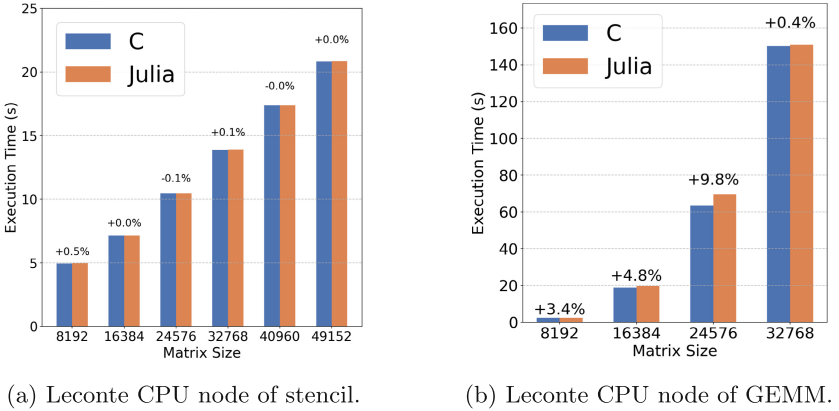


Fig. 3. End-to-end execution time of PaRSEC.jl compared with PaRSEC's official C examples on CPU.

Figure 3 reports end-to-end execution time on the Leconte CPU node for (a) stencil and (b) DTD GEMM, comparing PaRSEC.jl against the corresponding official PaRSEC C examples.

Stencil. For stencil, same as PyPaRSEC, PaRSEC.jl primarily acts as a driver that initializes the runtime, prepares data collections, and launches an existing PaRSEC taskpool; the task graph and kernels remain unchanged. As shown in Fig. 3(a), PaRSEC.jl matches the C baseline extremely closely across all tested problem sizes: the relative differences stay within $\pm 0.5\%$ (often near 0%).

DTD GEMM. We first examine the overhead of DTD task insertion. Table 2 shows that PaRSEC.jl incurs about +100% insertion overhead at 64 tasks, and the overhead increases with task-graph size, reaching +457% at 110,592 tasks.

For CPU GEMM, PaRSEC.jl exhibits modest slowdowns relative to the C baseline for most sizes, with the gap varying by problem size. As shown in Fig. 3(b), the overhead is +3.4% at 8192, increases to +4.8% at 16384, peaks at +9.8% at 24576, and then drops back to +0.4% at 32768.

6 Conclusion and Future Work

This paper presents two high-level language bindings to the PaRSEC task-based runtime: PyPaRSEC for Python and PaRSEC.jl for Julia. Both interfaces expose PaRSEC’s essential abstractions—context management, tiled data collections, task creation, and runtime execution—so users can build parallel workflows from productive languages while preserving the execution semantics of official PaRSEC examples. We evaluate the bindings with two representative workloads: stencil (PTG) and GEMM (DTD). The results show that the bindings act as thin drivers and track the native C baseline closely. For DTD GEMM, the bindings reproduce the official workflow structure and remain performance-competitive when compute is delegated to optimized backends (BLAS on CPUs and vendor GPU libraries). Overall, our study demonstrates that PaRSEC can be effectively leveraged from Python and Julia, bridging rapid prototyping and performance-oriented execution on heterogeneous systems. Going forward, we plan to (i) add AMD GPU support to PyPaRSEC, (ii) extend PaRSEC.jl with both NVIDIA and AMD GPU backends, and (iii) package a broader set of commonly used application kernels.

Acknowledgments. This research was supported by the U.S. National Science Foundation under Award OAC-2451577. For computational resources, this work utilized the compute node at the Innovative Computing Laboratory of the University of Tennessee, Knoxville. This research also used resources (Polaris cluster) of the Argonne Leadership Computing Facility, which is a U.S. Department of Energy Office of Science User Facility operated under contract DE-AC02-06CH11357.

References

1. Bosilca, G., Bouteiller, A., Danalis, A., Hérault, T., Lemarinier, P., Dongarra, J.J.: DAGuE: a generic distributed DAG engine for high performance computing. *Parallel Comput.* **38**(1–2), 37–51 (2012)
2. Bouteiller, A., Hérault, T., Cao, Q., Schuchart, J., Bosilca, G.: PaRSEC: scalability, flexibility, and hybrid architecture support for task-based applications in ECP. *Int. J. High Perform. Comput. Appl.* (2024)
3. Danalis, A., Bosilca, G., Bouteiller, A., Hérault, T., Dongarra, J.: PTG: an abstraction for unhindered parallelism. In: 2014 Fourth International Workshop on Domain-Specific Languages and High-Level Frameworks for High Performance Computing, pp. 21–30. IEEE (2014)

4. Bosilca, G., Robert, J., Harrison, T., Herault, M.M., Javanmard, P., Nookala, E.F.: Valeev: the template task graph (TTG)-an emerging practical dataflow programming paradigm for scientific simulation at extreme scale. In: IEEE/ACM 5th International Workshop on Extreme Scale Programming Models and Middleware (ESPM2). IEEE (2020)
5. Hoque, R., Herault, T., Bosilca, G., Dongarra, J.: Dynamic task discovery in PaRSEC: a data-flow task-based runtime. In: Proceedings of the 8th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems, Scala '17 (2017)
6. Harris, C.R., et al.: Array programming with numpy. *Nature* **585**(7825), 357–362 (2020)
7. Virtanen, P., et al.: Bright: scipy 1.0: fundamental algorithms for scientific computing in python. *Nat. Methods* **17**(3), 261–272 (2020)
8. Bezanson, J., Edelman, A., Karpinski, S., Shah, V.B.: Julia: a fresh approach to numerical computing. *SIAM Rev.* **59**(1), 65–98 (2017)
9. Paszke, A., et al.: Pytorch: an imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems, vol. 32 (2019)
10. Slaughter, E., Lee, W., Treichler, S., Bauer, M., Aiken, A.: Regent: a high-productivity programming language for HPC with logical regions. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–12 (2015)
11. OpenMP: OpenMP 4.5 Complete Specifications (2015)
12. Duran, A., Ferrer, R., Ayguade, E., Badia, R.M., Labarta, J.: A proposal to extend the OpenMP tasking model with dependent tasks. *Intl. J. Parallel Program.* **37**(3), 292–305 (2009)
13. Lordan, F., et al.: Servicess: an interoperable programming framework for the cloud. *J. Grid Comput.* (2014)
14. Augonnet, C., Thibault, S., Namyst, R., Wacrenier, P.: StarPU: a unified platform for task scheduling on heterogeneous multicore architectures. *Concurrency Computat. Pract. Exper.* **23**, 187–198 (2011)
15. Heller, T., Kaiser, H., Iglberger, K.: Application of the ParalleX execution model to stencil-based problems. *Comput. Sci. Res. Dev.* **28**(2–3), 253–261 (2013)
16. Bauer, M., Treichler, S., Slaughter, E., Aiken, A.: Legion: expressing locality and independence with logical regions. In: International Conference for High Performance Computing, Networking, Storage and Analysis, SC, pp. 1–11. IEEE (2012)
17. StarPU Project: StarPU release notes/news: The 1.4 release series brings a Python interface (2023). <https://starpup.gitlabpages.inria.fr/news.html>. Accessed 22 Jan 2026
18. Slaughter, E., Aiken, A.: Pygion: flexible, scalable task-based parallelism with Python. In: 2019 IEEE/ACM Parallel Applications Workshop, Alternatives To MPI (PAW-ATM) at SC19 (2019)
19. Cao, Q., et al.: Performance analysis of tile low-rank cholesky factorization using parsec instrumentation tools. In: 2019 IEEE/ACM International Workshop on Programming and Performance Visualization Tools (ProTools), pp. 25–32. IEEE (2019)
20. Cao, Q., Bosilca, G., Losada, N., Wei, W., Zhong, D., Dongarra, J.: Evaluating data redistribution in parsec. *IEEE Trans. Parallel Distrib. Syst.* **33**(8), 1856–1872 (2021)
21. Cao, Q., Abdulah, S., Ltaief, H., Genton, M.C., Keyes, D., Bosilca, G.: Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion. In: 2023 IEEE International Conference on Cluster Computing (CLUSTER), pp. 330–342. IEEE (2023)

22. Cao, Q., et al.: Leveraging parsec runtime support to tackle challenging 3D data-sparse matrix problems. In: 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 79–89. IEEE (2021)
23. Cao, Q., et al.: A framework to exploit data sparsity in tile low-rank cholesky factorization. In: 2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 414–424. IEEE (2022)
24. Zhang, Q., Alomairy, R., Wang, D., Zhuowei, G., Cao, Q.: High-performance mixed-precision matrix multiplication via tile-centric design on modern architectures. *SN Comput. Sci.* **7**(1), 24 (2026)
25. Abdulah, S., et al.: Boosting earth system model outputs and saving petabytes in their storage using exascale climate emulators. In: ACM Gordon Bell Prize for Climate Modelling Finalist (2024)
26. Ltaief, H., et al.: Toward capturing genetic epistasis from multivariate genome-wide association studies using mixed-precision kernel ridge regression. In: ACM Gordon Bell Prize Finalist (2024)
27. Cao, Q., et al.: Reshaping geostatistical modeling and prediction for extreme-scale environmental applications. In: SC22: International Conference for High Performance Computing, Networking, Storage and Analysis (ACM Gordon Bell Prize Finalist), pp. 1–12. IEEE (2022)
28. Ranocha, H., Schlottke-Lakemper, M., Winters, A.R., Faulhaber, E., Chan, J., Gassner, G.J.: Adaptive numerical simulations with Trixi. JI: a case study of Julia for scientific computing. ArXiv Preprint [arXiv:2108.06476](https://arxiv.org/abs/2108.06476) (2021)
29. Berman, E., Ginesin, J.: The state of julia for scientific machine learning, (2024). [arXiv:2410.10908](https://arxiv.org/abs/2410.10908) arXiv preprint
30. Nazarathy, Y., Klok, H.: Statistics with Julia: Fundamentals for Data Science, Machine Learning and Artificial Intelligence. Springer, Cham (2021). <https://doi.org/10.1007/978-3-030-70901-3>
31. Mogensen, P., Riseth, A.: Optim: a mathematical optimization package for Julia. *J. Open Source Softw.* **3**(24) (2018)
32. Landeros, A., et al.: Biosimulator. JL: stochastic simulation in Julia. *Comput. Methods Programs Biomed.* **167**, 23–35 (2018)